TRƯỜNG: **ĐH CT TP.HCM** KHOA: **CÔNG NGHỆ THÔNG TIN** BỘ MÔN: **KHDL** MH: **TH HỌC MÁY**	**BUỔI 2. PHÂN LỚP (KNN_SVM)**	

A. MỤC TIÊU

- Xác định được bài toán phân lớp dựa trên dữ liệu thu thập được.
- Sử dụng thuật toán phân lớp phù hợp với bộ dữ liệu cụ thể.
- Đánh giá thực trạng bộ dữ liệu.
- Tiền xử lý dữ liệu (nếu có).
- Cài đặt thuật toán huấn luyện mô hình phân lớp cho bộ dữ liệu cụ thể: chia dữ liệu, chọn các siêu tham số,…).
- Đánh giá mô hình thông qua các độ đo.
- Các cách tinh chỉnh mô hình.
- Ứng dụng mô hình đã huấn luyện được.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

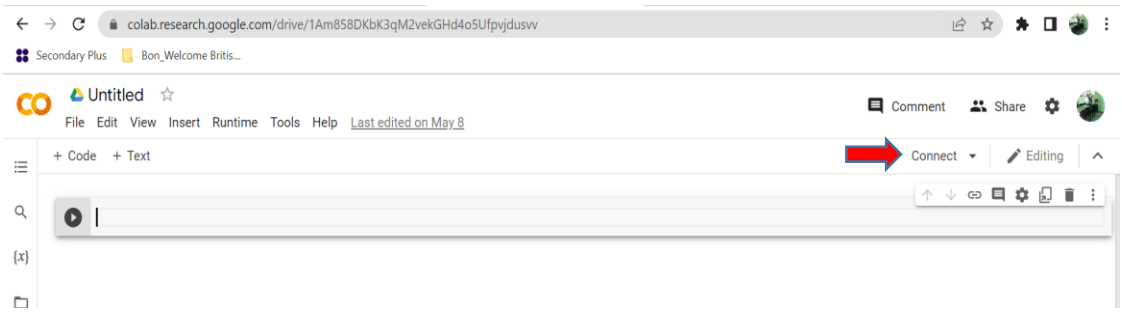
C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. **Sử dụng Google Colab để viết chương trình:**

   B1. Truy cập địa chỉ: https://colab.research.google.com/

   B2. Vào file → new notebook



   B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. **KNN (K-Nearest Neighbors)**

KNN là một thuật toán máy học dùng cho cả phân loại và hồi quy. Thuật toán này dựa trên giả định rằng những điểm dữ liệu gần nhau trong không gian đặc trưng sẽ có xu hướng thuộc cùng một nhãn hoặc giá trị mục tiêu tương tự nhau.

Nguyên lý hoạt động của KNN:

- Xác định tham số K: Chọn số lượng hàng xóm gần nhất (K) để xem xét khi ra quyết định.

- Tính khoảng cách: Đo khoảng cách từ điểm cần dự đoán tới tất cả các điểm khác trong tập dữ liệu (khoảng cách Euclidean thường được sử dụng).

- Tìm K điểm gần nhất: Sắp xếp các điểm theo khoảng cách, sau đó chọn K điểm gần nhất.

Phân loại hoặc hồi quy:

- Đối với phân loại: Lấy nhãn mà phần lớn các điểm trong K điểm gần nhất.
- Đối với hồi quy: Tính trung bình giá trị mục tiêu của K điểm gần nhất.

Ưu điểm và Nhược điểm của KNN:

Ưu điểm:

- Đơn giản và dễ hiểu.
- Hiệu quả với dữ liệu có mối quan hệ phi tuyến.

Nhược điểm:

- Nhạy cảm với dữ liệu nhiễu và các đặc trưng không quan trọng.
- Yêu cầu bộ nhớ lớn khi xử lý với dữ liệu có số lượng mẫu lớn.
- Phụ thuộc vào việc chọn K thích hợp và phương pháp tính khoảng cách.

Ví dụ minh họa: Phân loại loài hoa Iris bằng KNN, tập dữ liệu Iris (có thể lấy từ thư viện sklearn hoặc Kaggle).

Dữ liệu: Tập dữ liệu Iris gồm có 150 mẫu với 4 đặc trưng

Chiều dài đài hoa (sepal length)

Chiều rộng đài hoa (sepal width)

Chiều dài cánh hoa (petal length)

Chiều rộng cánh hoa (petal width)

Mỗi mẫu thuộc về một trong ba loài (3 loại): Setosa, Versicolor, hoặc Virginica.

Code mẫu các bước thực hiện:

```
# Thư viện

    import numpy as np

    import pandas as pd

    from sklearn.datasets import load_iris

    from sklearn.model_selection import train_test_split

    from sklearn.neighbors import KNeighborsClassifier

    from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

    import matplotlib.pyplot as plt

    import seaborn as sns

# Tải dữ liệu Iris

    iris = load_iris()

    X = iris.data  # Dữ liệu đặc trưng

    y = iris.target  # Nhãn mục tiêu

# Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Xây dựng mô hình KNN và chọn K=3

    k = 3

    knn_model = KNeighborsClassifier(n_neighbors=k)

    knn_model.fit(X_train, y_train)  # Huấn luyện mô hình
```

```
# Dự đoán trên tập kiểm tra
    y_pred = knn_model.predict(X_test)
#Đánh giá mô hình
    accuracy = accuracy_score(y_test, y_pred)
    conf_matrix = confusion_matrix(y_test, y_pred)
    print(f"Độ chính xác của mô hình KNN với K={k}: {accuracy:.2f}")
    print("Ma trận nhầm lẫn:")
    print(conf_matrix)
    print("Báo cáo phân loại:")
    print(classification_report(y_test, y_pred))
# Ma trận nhầm lẫn
    plt.figure(figsize=(8, 6))
    sns.heatmap(conf_matrix, annot=True, cmap="YlGnBu", fmt="d",
     xticklabels=iris.target_names, yticklabels=iris.target_names)
    plt.xlabel('Dự đoán')
    plt.ylabel('Thực tế')
    plt.title(f"Ma trận nhầm lẫn của mô hình KNN với K={k}")
    plt.show()
```

3. SVM (Support Vector Machine)

SVM là một thuật toán máy học mạnh mẽ, chủ yếu dùng cho bài toán phân loại và hồi quy. SVM là một mô hình giám sát hoạt động bằng cách tìm một siêu phẳng phân tách dữ liệu theo nhãn mục tiêu, với mục tiêu tạo ra khoảng cách tối đa giữa các lớp dữ liệu khác nhau.

Ý tưởng chính của SVM

- Siêu phẳng phân tách: SVM cố gắng tìm một siêu phẳng tốt nhất (hyperplane) để phân tách các điểm dữ liệu thuộc các lớp khác nhau trong không gian đặc trưng.
- Tối đa hóa khoảng cách: Khoảng cách giữa siêu phẳng và các điểm dữ liệu gần nhất của từng lớp (gọi là các support vectors) được tối đa hóa.
- Nhân (Kernel): SVM có thể sử dụng các hàm Kernel (hạt nhân) như RBF (Radial Basis Function), Linear, Polynomial,... để phân loại dữ liệu phi tuyến, tức là chuyển đổi dữ liệu vào một không gian mới giúp phân tách dễ dàng hơn.

Ví dụ minh họa sử dụng SVM trên bộ dữ liệu Breast Cancer từ thư viện sklearn:

- Bộ dữ liệu này bao gồm các thông tin về tế bào ung thư vú, và yêu cầu là phân loại các mẫu thành lành tính hoặc ác tính.

Code minh họa:

```
# Thư viện

    import numpy as np

    import pandas as pd

    from sklearn import datasets

    from sklearn.model_selection import train_test_split

    from sklearn.svm import SVC

    from sklearn.preprocessing import StandardScaler

    from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

    import seaborn as sns

    import matplotlib.pyplot as plt

# Tải bộ dữ liệu Breast Cancer

    cancer = datasets.load_breast_cancer()

    X = cancer.data  # Dữ liệu đặc trưng

    y = cancer.target  # Nhãn mục tiêu (0: Ác tính, 1: Lành tính)

# Chia dữ liệu thành tập huấn luyện và tập kiểm tra
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Chuẩn hóa dữ liệu

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Xây dựng mô hình SVM với kernel RBF

svm_model = SVC(kernel='rbf', C=1, gamma='scale')  # C và gamma là các siêu
tham số

svm_model.fit(X_train, y_train)  # Huấn luyện mô hình

# Dự đoán trên tập kiểm tra

y_pred = svm_model.predict(X_test)

# Đánh giá mô hình

accuracy = accuracy_score(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)

print(f"Độ chính xác của mô hình SVM: {accuracy:.2f}")

print("Ma trận nhầm lẫn:")

print(conf_matrix)

print("Báo cáo phân loại:")

print(classification_report(y_test, y_pred))

# Trực quan hóa ma trận nhầm lẫn

plt.figure(figsize=(8, 6))

sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
xticklabels=['Malignant', 'Benign'], yticklabels=['Malignant', 'Benign'])
```

```
plt.xlabel("Giá trị dự đoán")

plt.ylabel("Giá trị thực tế")

plt.title("Ma trận nhầm lẫn của mô hình SVM")

plt.show()
```

III. Bài tập ở lớp

Bài 1.Thực nghiệm lại code mẫu KNN với các yêu cầu:

1. Thực nghiệm với các giá trị K khác nhau để tìm ra giá trị K tối ưu.
2. Chuẩn hóa dữ liệu đầu vào để tránh ảnh hưởng của các đặc trưng có đơn vị hoặc phạm vi khác nhau.
3. Thực nghiệm với các đơn vị tính khoảng cách khác nhau như khoảng cách Manhattan, Minkowski để xem sự thay đổi trong hiệu quả của mô hình.

Bài 2. Thuật toán KNN với tập dữ liệu giá nhà

Mô tả: Sử dụng KNN (dạng hồi quy) để dự đoán giá nhà trên tập dữ liệu Boston Housing.

Yêu cầu:

1. Tải tập dữ liệu Boston Housing.
2. Phân chia dữ liệu thành tập huấn luyện và tập kiểm tra.
3. Chuẩn hóa dữ liệu để các đặc trưng ở cùng một khoảng giá trị.
4. Áp dụng KNN Regressor với các giá trị k khác nhau (ví dụ: 3, 5, 7, 9).
5. Tính toán các chỉ số MSE (Mean Squared Error) và R^2 score để đánh giá hiệu quả mô hình cho từng giá trị của k.

Bài 3. Sử dụng thuật toán SVM xây dựng mô hình phân lớp hoa với bộ dữ liệu hoa Iris. Chú ý thực nghiệm với nhiều cách chia dữ liệu khác nhau, so sánh đánh giá kết quả đạt được của mỗi cách chia dữ liệu trên bộ dữ liệu này.

Bài 4. Thực hiện lại bài 2 với thuật toán SVM dạng hồi quy (SVR trường hợp phân lớp là SVC) với kernel là linear và rbf.

IV. Bài tập về nhà

Bài 1. Thực nghiệm các giá trị khác nhau kernel đánh giá hiệu suất của mô hình với SVM.

Bài 2. Tìm hiểu sử dụng GridSearchCV để tìm siêu tham số tối ưu cho mô hình SVM.

Bài 3. Dùng KNN, SVM với tập dữ liệu chữ số

Mô tả: Sử dụng lần lượt KNN, SVM để phân loại chữ số trong tập dữ liệu Digits.

Yêu cầu:

1. Tải tập dữ liệu Digits từ sklearn.datasets.
2. Chuẩn hóa dữ liệu và phân chia thành tập huấn luyện và tập kiểm tra.
3. Huấn luyện lần lượt mô hình KNN, SVM (với kernel là poly và rbf).
4. So sánh độ chính xác của hai mô hình và hiển thị ma trận nhầm lẫn (confusion matrix) để đánh giá hiệu quả mô hình.